

RAG KnowledgeNexus

软件设计说明书

概要设计 | 模块交互 | 详细设计 | 数据结构 | 接口与部署

版本	v1.0
日期	2026-09-21
阶段	阶段 3 软件设计
状态	设计评审稿

目录

1. 文档说明	6
2. 设计目标	6
3. 技术选型	6
3.1 后端	6
3.2 前端	7
3.3 AI 与检索	7
4. 总体架构	7
4.1 分层架构	7
4.2 后端模块划分	7
4.3 业务模块划分	7
4.4 前端模块划分	8
5. 模块交互设计	8
5.1 普通请求链路	8
5.2 文档处理链路	9
5.3 RAG 问答链路	9
5.4 删除一致性链路	9
6. 详细设计	9
6.1 认证与用户模块	9
6.1.1 模块职责	9
6.1.2 登录流程	9
6.1.3 Token 设计	10
6.1.4 用户数据结构	10
6.1.5 业务规则	10
6.1.6 接口设计	10
6.2 部门模块	11
6.2.1 模块职责	11
6.2.2 数据结构	11
6.2.3 树构建逻辑	11
6.2.4 删除规则	11
6.2.5 接口设计	11
6.3 知识库模块	12
6.3.1 模块职责	12
6.3.2 数据结构	12
6.3.3 权限算法	12
6.3.4 删除策略	12
6.3.5 接口设计	13
6.4 文档模块	13
6.4.1 模块职责	13
6.4.2 数据结构	13
6.4.3 上传校验	13
6.4.4 上传步骤	14
6.4.5 删除与重解析	14

6.4.6 接口设计	14
6.5 解析与分片模块	14
6.5.1 模块职责	14
6.5.2 解析适配器	14
6.5.3 分片算法	15
6.5.4 数据处理状态	15
6.5.5 分片数据结构	15
6.6 向量化与 Qdrant 模块	15
6.6.1 模块职责	15
6.6.2 Embedding 设计	15
6.6.3 Qdrant 数据结构	16
6.6.4 检索算法	16
6.6.5 删除算法	16
6.7 智能问答模块	16
6.7.1 模块职责	16
6.7.2 对话与消息数据结构	16
6.7.3 系统提示词	17
6.7.4 低置信度处理	17
6.7.5 接口设计	17
6.8 工单模块	17
6.8.1 模块职责	18
6.8.2 工单数据结构	18
6.8.3 状态流转	18
6.8.4 接口设计	18
6.9 统计模块	18
6.9.1 模块职责	18
6.9.2 日统计数据结构	19
6.9.3 热门问题算法	19
6.9.4 定时任务	19
6.9.5 接口设计	19
6.10 前端详细设计	19
6.10.1 路由与权限	19
6.10.2 请求封装	20
6.10.3 状态管理	20
6.10.4 列表页面设计	20
7. 数据设计	20
7.1 实体关系	20
7.2 主要数据表	20
7.3 ID 与时间	21
7.4 对象存储	21
7.5 向量存储	21
8. 接口通用设计	21
8.1 路径与协议	21
8.2 认证	21

8.3 统一响应	22
8.4 分页响应	22
8.5 通用校验	22
9. 接口详细设计	22
9.1 登录	22
9.2 注册	23
9.3 知识库查询	23
9.4 创建知识库	23
9.5 上传文档	23
9.6 获取文档预览地址	24
9.7 创建对话	24
9.8 提问	24
9.9 提交反馈	24
9.10 创建工单	24
9.11 解决工单	25
9.12 每日统计	25
10. 接口错误设计	25
10.1 HTTP 状态	25
10.2 业务码	25
10.3 异常处理	26
11. 安全设计	26
11.1 认证与授权	26
11.2 密码与 Token	26
11.3 文件安全	26
11.4 数据安全	26
12. 异步与定时任务设计	27
12.1 线程池	27
12.2 任务状态	27
12.3 重试策略	27
12.4 定时统计	27
13. 日志与监控设计	27
13.1 日志分类	27
13.2 监控指标	28
13.3 健康检查	28
14. 部署设计	28
14.1 运行拓扑	28
14.2 配置隔离	28
14.3 发布流程	28
15. 测试设计	29
15.1 单元测试	29
15.2 集成测试	29
15.3 接口测试	29
15.4 前端测试	29
15.5 性能测试	29

16. 实施顺序

30

16.1 第一阶段：基础能力

30

16.2 第二阶段：RAG 主链路

30

16.3 第三阶段：业务闭环

30

16.4 第四阶段：工程化与增强

30

17. 设计追踪

30

1. 文档说明

本说明书用于指导 RAG KnowledgeNexus 的编码实现。文档先给出概要设计，确定系统模块、技术选型和模块交互；再给出详细设计，说明各模块内部逻辑、数据结构、接口契约、异常处理和状态流转。

设计遵循以下原则：

- 业务分层清晰，Controller 不承载核心业务规则。
- 知识库是数据权限和向量检索的隔离单位。
- 文档处理采用异步任务，避免阻塞上传请求。
- 解析、向量化、索引三个阶段的成功状态必须可区分。
- 所有资源访问执行统一的归属和可见范围校验。
- 外部服务调用具备超时、错误处理和重试路径。
- 接口使用版本化前缀和统一响应结构。

2. 设计目标

目标	设计措施
支持多种文档	PDFBox、Apache POI 和 UTF-8 文本读取适配器
支持语义检索	Embedding + Qdrant Collection + Payload Filter
支持 RAG 问答	向量检索、上下文拼接、系统提示词、来源返回
支持资源隔离	用户、部门、角色和知识库可见范围组合授权
支持异步处理	文档解析队列、向量化队列、失败重试
支持运营分析	问答消息落库、反馈评分、定时统计
支持前后端分离	Vue 3 SPA + Spring Boot REST API
支持可部署	dev/prod 配置隔离，外部依赖环境变量化

3. 技术选型

3.1 后端

技术	版本或方案	用途
Java	17	后端开发语言
Spring Boot	2.7.18	应用框架
Spring Security	Spring Boot 依赖	认证和角色授权
JWT	JJWT	无状态登录凭证
MyBatis-Plus	3.5.3.1	ORM、分页和 CRUD
MySQL	8.0	业务数据存储
Redis	6.x 或以上	缓存、限流和短期状态
MinIO	8.x 客户端	文档与头像对象存储
PDFBox	3.x	PDF 文本提取
Apache POI	5.x	DOCX 文本和表格提取
OkHttp	4.x	Embedding、Chat 和 Qdrant HTTP 调用
Knife4j	3.x	接口调试文档

3.2 前端

技术	用途
Vue 3	单页应用框架
TypeScript	类型约束
Vue Router	页面路由
Pinia	用户和界面状态管理
Element Plus	基础组件
Axios	HTTP 请求和拦截器
ECharts	工作台统计图表

3.3 AI 与检索

组件	设计
Embedding 模型	阿里云 DashScope text-embedding-v3, 1024 维
Chat 模型	DashScope qwen-plus
向量数据库	Qdrant, Cosine 距离, HNSW 索引
Collection	每个知识库一个 kb_{kbld}
检索范围	通过 kb_id Payload 过滤

4. 总体架构

4.1 分层架构



4.2 后端模块划分

模块	职责
rag-doc-common	常量、枚举、JWT、MinIO、通用属性
rag-doc-model	POJO、DTO、VO、统一响应和业务码
rag-doc-server	Controller、Service、Mapper、Config、Task

4.3 业务模块划分

业务模块	核心服务	主要数据
认证与用户	UserLoginService、UserService	用户、角色、状态
部门	DepartmentService	部门树、成员
知识库	KnowledgeService、KbFavoriteService	知识库、收藏
文档	DocumentService	文档元数据、原文件
解析分片	DocumentParseService、ChunkService	文本分片
向量化	EmbeddingService、VectorizationService	向量批次
向量索引	QdrantIndexService	Collection、Point
问答	QaService、ChatService	对话、消息、引用
工单	TicketOrderService	工单和回复
统计	StatService、StatScheduledTask	日统计、热门问题

4.4 前端模块划分

页面模块	路由	状态
登录注册	/login	登录表单、注册表单
工作台	/dashboard	概览指标、最近文档、趋势
知识库	/knowledgeBase	列表、创建、编辑、详情入口
知识库详情	/knowledgeBase/:id	文档列表、上传、预览
文档	/document	全量文档筛选和管理
问答	/qa	知识库选择、会话、消息
工单	/ticket	工单筛选和管理员处理
用户	/user	用户分页和状态管理
部门	/department	部门树和成员
个人中心	/profile	资料、密码和头像

5. 模块交互设计

5.1 普通请求链路

- 页面
- > Axios 添加 Authorization

-> Spring Security JWT Filter

-> Controller

-> Service

-> PermissionService

-> Mapper

-> MySQL

-> ResponseResult

-> 页面

5.2 文档处理链路

DocumentController

- > DocumentService.upload
- > 权限校验
- > MinIO 上传
- > doc_document 记录 PENDING
- > 解析队列
- > DocumentParseService
- > doc_chunk 批量保存
- > 向量化队列
- > EmbeddingService
- > QdrantIndexService
- > 更新分片和文档最终状态

5.3 RAG 问答链路

QaController

- > QaService
- > 校验对话归属和知识库权限
- > EmbeddingService.embed(question)
- > QdrantIndexService.search(kbld, vector, topK)
- > ChunkMapper 查询正文
- > ChatService.chat(systemPrompt, context, question)
- > 保存 QaMessage
- > 返回 answer 和 sources

5.4 删除一致性链路

删除请求

- > 校验资源归属
- > 标记数据库逻辑删除
- > 删除或软删除分片
- > 删除 Qdrant Point 或 Collection
- > 删除 MinIO 对象
- > 写入操作日志
- > 返回成功

任何外部步骤失败时写入补偿任务，由定时任务继续清理。

6. 详细设计

6.1 认证与用户模块

6.1.1 模块职责

- 注册、登录、登出和 Token 刷新。
- 获取和更新当前用户资料。
- 上传头像。
- 超级管理员查询、启停和删除用户。

6.1.2 登录流程

- 1 Controller 接收用户名和密码。
- 2 Service 根据用户名查询用户。
- 3 校验用户存在、状态正常且未删除。

- 4 使用 BCrypt 比较密码。
- 5 生成 JWT，载荷至少包含用户 ID、签发时间和过期时间。
- 6 返回 Token 和脱敏用户信息。

6.1.3 Token 设计

项目	设计
Token 类型	Bearer JWT
签名算法	HS256
默认有效期	1 小时
请求头	Authorization: Bearer <token>
刷新条件	Token 未过期且剩余时间低于刷新阈值
注销策略	客户端删除；服务端可通过 Redis 黑名单增强

6.1.4 用户数据结构

字段	类型	说明
id	Long	雪花 ID
username	String	登录名，唯一
password	String	BCrypt 哈希
nickname	String	显示名
email	String	邮箱
avatarUrl	String	头像对象路径
role	Integer	0 超管，1 知识库管理员，2 普通员工
deptId	Long	部门 ID
status	Integer	0 禁用，1 正常
lastLoginTime	DateTime	最近登录时间
isDeleted	Integer	逻辑删除

6.1.5 业务规则

- 公开注册固定普通员工角色，忽略客户端传入角色。
- 用户只能修改自己的资料。
- 密码修改必须验证旧密码。
- 禁用或删除用户的 Token 不允许继续访问。
- 超级管理员不能被普通管理员启用、禁用或删除。

6.1.6 接口设计

方法	路径	说明	权限
POST	/api/v1/user/login	登录	公开
POST	/api/v1/user/register	注册	公开
POST	/api/v1/user/refresh	刷新 Token	公开路径，需 Token
GET	/api/v1/user/info	获取当前用户	登录
POST	/api/v1/user/logout	登出	登录
POST	/api/v1/user/update	更新当前用户	登录

方法	路径	说明	权限
POST	/api/v1/user/avatar	上传头像	登录
GET	/api/v1/user/page	用户分页	超级管理员
DELETE	/api/v1/user/delete/{id}	删除用户	超级管理员
POST	/api/v1/user/status	设置用户状态	超级管理员

6.2 部门模块

6.2.1 模块职责

- 维护部门树。
- 查询部门成员。
- 为知识库部门可见范围提供部门数据。

6.2.2 数据结构

字段	类型	说明
id	Long	部门 ID
name	String	部门名称
parentId	Long	父部门 ID
sort	Integer	同级排序
remark	String	备注
isDeleted	Integer	逻辑删除

6.2.3 树构建逻辑

- 查询全部未删除部门。
- 按 parentId 建立子节点索引。
- 将 parentId 为空、为 0、等于自身或指向不存在部门的节点视为根节点。
- 使用队列或递归构建树。
- 统计每个部门的用户数。

6.2.4 删除规则

- 存在未删除子部门时禁止删除。
- 存在未删除用户时禁止删除。
- 删除采用逻辑删除。

6.2.5 接口设计

方法	路径	说明	权限
GET	/api/v1/department/tree	部门树	登录
GET	/api/v1/department/list	平铺列表	登录
GET	/api/v1/department/{deptId}/members	部门成员	登录
POST	/api/v1/department/create	创建部门	超级管理员
POST	/api/v1/department/update	修改部门	超级管理员

方法	路径	说明	权限
DELETE	/api/v1/department/delete/{id}	删除部门	超级管理员

6.3 知识库模块

6.3.1 模块职责

- 创建、查询、修改和删除知识库。
- 控制私有、部门和全员三级可见范围。
- 维护收藏关系。
- 为文档和问答提供访问权限判断。

6.3.2 数据结构

字段	类型	说明
id	Long	知识库 ID
name	String	名称，唯一
description	String	描述
ownerId	Long	所有者
visibility	Integer	0 私有，1 部门，2 全员
deptId	Long	部门 ID
docCount	Integer	文档数量
embedModel	String	向量模型
isDeleted	Integer	逻辑删除

6.3.3 权限算法

可以查看知识库 =
 用户是超级管理员
 或 用户是知识库所有者
 或 visibility = 全员
 或 visibility = 部门且用户部门等于知识库部门

可以管理知识库 =
 用户是超级管理员
 或 用户是知识库所有者

6.3.4 删除策略

知识库删除必须按顺序执行：

- 1 逻辑删除知识库。
- 2 逻辑删除该知识库下的文档。
- 3 逻辑删除或物理删除分片。
- 4 删除 Qdrant Collection。
- 5 按策略删除 MinIO 对象。
- 6 软删除收藏记录。
- 7 记录操作日志和补偿任务。

6.3.5 接口设计

方法	路径	说明	权限
GET	/api/v1/user/knowledge-base/list	分页查询	登录
GET	/api/v1/user/knowledge-base/{id}	详情	登录且可查看
POST	/api/v1/user/knowledge-base/create	创建	超管或知识库管理员
POST	/api/v1/user/knowledge-base/update	修改	超管或所有者
PUT	/api/v1/user/knowledge-base/{kbld}/favorite	收藏切换	登录且可查看
DELETE	/api/v1/user/knowledge-base/{kbld}	删除	超管或所有者
GET	/api/v1/user/knowledge-base/name	名称列表	登录，必须按权限过滤

6.4 文档模块

6.4.1 模块职责

- 文档上传、分页、详情、预览、改名、删除和重新解析。
- 管理文档元数据与 MinIO 对象的映射。
- 触发解析和索引任务。

6.4.2 数据结构

字段	类型	说明
id	Long	文档 ID
kbld	Long	知识库 ID
title	String	文档标题
fileName	String	原始文件名
fileType	String	文件扩展名
fileSize	Long	字节数
minioPath	String	对象路径或标准地址
parseStatus	Integer	文档处理状态
parseError	String	失败原因
chunkCount	Integer	分片数
uploadUserId	Long	上传用户
wordCount	Integer	正文字符数
isDeleted	Integer	逻辑删除

6.4.3 上传校验

- 用户必须登录。
- 用户必须对目标知识库具有管理权限。
- 文件名必须包含允许的扩展名。
- 扩展名必须属于 pdf、docx、doc、txt。
- 文件不能为空，最大 50 MB。
- MIME 类型和文件头应与扩展名匹配。

6.4.4 上传步骤

- 1 校验参数、权限和文件。
- 2 生成 UUID 对象名。
- 3 上传原文件到 MinIO。
- 4 写入文档元数据，状态为待处理。
- 5 发布解析任务。
- 6 立即返回上传成功。

6.4.5 删除与重解析

- 删除文档时清理数据库、Qdrant 和 MinIO。
- 重解析前删除旧分片和旧向量。
- 重解析期间文档状态必须变为处理中。
- 旧索引未被清理前不得写入新版本向量。

6.4.6 接口设计

方法	路径	说明	权限
GET	/api/v1/document/documents	分页查询	登录且资源可见
POST	/api/v1/document/uploadDocument	上传	知识库管理权限
GET	/api/v1/document/{id}	详情	知识库查看权限
GET	/api/v1/document/documents/preview/{doc_id}	预览地址	知识库查看权限
POST	/api/v1/document/updateDocumentTitle/{doc_id}	修改标题	上传者或管理员
DELETE	/api/v1/document/{doc_id}	删除	上传者或管理员
POST	/api/v1/document/{doc_id}/reparse	重新解析	上传者或管理员
GET	/api/v1/document/kb/{kb_id}	知识库文档	知识库查看权限

6.5 解析与分片模块

6.5.1 模块职责

- 从 MinIO 读取文档。
- 按文件类型提取文本。
- 对文本进行清洗和语义分片。
- 保存分片并触发向量化。

6.5.2 解析适配器

文件类型	解析方式	输出
PDF	PDFBox PDFTextStripper	页面文本
DOCX	XWPFDocument	段落和表格文本
DOC	建议先转换为 DOCX 或使用兼容解析器	正文
TXT	UTF-8 字节读取	纯文本

6.5.3 分片算法

- 按双换行切分自然段。
- 合并连续空白和全角空格。
- 段落不超过 500 字符时独立成片。
- 超长段落按中英文句末标点拆句。
- 累积句子，超过 500 字符时从后向前寻找标点切分。
- 新分片保留上一片末尾约 50 字符作为重叠。
- 记录 chunk_index、page_num 和内容长度。

6.5.4 数据处理状态

状态	含义
PENDING	已上传，等待解析
PARSING	正在提取文本和分片
INDEXING	分片已保存，等待向量化
SUCCESS	解析、向量化和索引全部成功
FAILED	任一步骤失败且重试耗尽

6.5.5 分片数据结构

字段	类型	说明
id	Long	分片 ID，也是 Qdrant Point ID
docId	Long	文档 ID
kbId	Long	知识库 ID
chunkIndex	Integer	文档内顺序
content	Text	分片正文
contentLength	Integer	字符数
vectorId	String	向量 ID
embedStatus	Integer	待处理、成功、失败
pageNum	Integer	页码或近似位置

6.6 向量化与 Qdrant 模块

6.6.1 模块职责

- 批量生成文本向量。
- 按知识库创建 Collection。
- 写入、检索和删除向量。
- 维护分片向量状态。

6.6.2 Embedding 设计

项目	设计
模型	text-embedding-v3
维度	1024
批量大小	建议 10-25 条，根据服务限制配置
输入	分片正文或用户问题

项目	设计
输出	float[1024]
失败策略	记录失败状态，支持批量重试

6.6.3 Qdrant 数据结构

元素	值
Collection	kb_{kbld}
Point ID	chunk.id
距离	Cosine
Payload	chunk_id、kb_id、doc_id、chunk_index、content_preview

6.6.4 检索算法

- 1 将问题生成向量。
- 2 在对应知识库 Collection 中检索。
- 3 添加 kb_id 过滤条件。
- 4 返回得分最高的 Top-K Point。
- 5 根据分片 ID 查询正文。
- 6 保持检索得分顺序，供问答和引用展示。

6.6.5 删除算法

- 删除知识库时删除整个 Collection。
- 删除文档时按 doc_id 过滤删除 Point。
- 删除分片时删除对应 Point ID。
- 删除失败写入补偿任务表。

6.7 智能问答模块

6.7.1 模块职责

- 创建和管理对话。
- 接收问题并执行 RAG。
- 保存回答、引用、Token 和响应耗时。
- 接收用户反馈。

6.7.2 对话与消息数据结构

对话

字段	类型	说明
id	Long	对话 ID
userId	Long	所属用户
kbld	Long	知识库
title	String	对话标题

字段	类型	说明
isDeleted	Integer	逻辑删除

消息

字段	类型	说明
id	Long	消息 ID
conversationId	Long	对话 ID
userId	Long	用户 ID
question	Text	问题
answer	Text	回答
sourceChunks	JSON	引用分片
qualityScore	Integer	1 好评, -1 差评
tokensUsed	Integer	Token 数
llmModel	String	模型名称
searchType	String	检索类型
responseMs	Long	响应耗时

6.7.3 系统提示词

系统提示词必须约束模型：

- 只依据给定资料回答。
- 资料不足时明确说明。
- 不得编造事实和引用。
- 多个要点时分段回答。
- 回答保持简洁、准确。

6.7.4 低置信度处理

设计建议设置相似度阈值：

- 所有结果低于阈值时不调用大模型，直接返回资料不足。
- 部分结果低于阈值时过滤后再生成。
- 返回来源时标记相似度，便于用户判断。

6.7.5 接口设计

方法	路径	说明	权限
POST	/api/v1/qa/conversation/create	创建对话	登录且知识库可见
POST	/api/v1/qa/ask	提问	对话所有者
GET	/api/v1/qa/conversation/list	对话列表	登录，仅本人
GET	/api/v1/qa/conversation/detail/{conversationId}	对话详情	对话所有者
POST	/api/v1/qa/feedback	提交反馈	消息所有者

6.8 工单模块

6.8.1 模块职责

- 创建工单。
- 根据角色返回工单列表。
- 管理员回复并关闭工单。
- 为统计模块提供转工单数量。

6.8.2 工单数据结构

字段	类型	说明
id	Long	工单 ID
messageId	Long	关联消息
userId	Long	创建用户
title	String	标题
description	Text	描述
status	Integer	状态
assigneeId	Long	处理人
priority	Integer	优先级
kbId	Long	关联知识库
reply	Text	处理回复
replyTime	DateTime	回复时间
resolvedTime	DateTime	解决时间

6.8.3 状态流转

- 待处理(0)
- > 处理中(1)
- > 已解决(2)
- > 已关闭(3)

允许管理员直接从待处理转为已解决；已解决和已关闭工单禁止重复处理。

6.8.4 接口设计

方法	路径	说明	权限
POST	/api/v1/ticket/create	创建工单	登录
GET	/api/v1/ticket/list	工单列表	登录，按角色过滤
PUT	/api/v1/ticket/{id}/resolve	解决工单	超管或对应知识库管理员

6.9 统计模块

6.9.1 模块职责

- 统计每日问答量、活跃用户和好评率。
- 统计工单转人工数量。
- 统计模型平均响应时间。
- 统计新增文档数。

- 生成热门问题。

6.9.2 日统计数据结构

字段	类型	说明
statDate	Date	统计日期
totalQa	Integer	问答数
totalUsers	Integer	活跃用户数
positiveRate	Decimal	好评率
transferCount	Integer	工单数
avgResponseMs	Integer	平均响应耗时
newDocs	Integer	新增文档数

6.9.3 热门问题算法

- 获取指定日期问答消息。
- 将问题去除首尾空白、压缩空格并转小写。
- 使用归一化文本进行分组。
- 计算提问次数和质量分。
- 按提问次数降序排序。
- 按知识库维度写入热门问题表。

6.9.4 定时任务

项目	设计
执行时间	每日 02:00
统计日期	前一日
幂等键	stat_date 或 question_hash + kb_id
失败处理	记录日志，允许人工重新执行

6.9.5 接口设计

方法	路径	说明	权限
GET	/api/v1/stat/daily	每日统计	超级管理员
GET	/api/v1/stat/hot-questions	热门问题	超级管理员

6.10 前端详细设计

6.10.1 路由与权限

路由守卫检查本地 Token：

- 未登录用户只能访问登录页。
- 已登录用户访问登录页时跳转工作台。
- 管理员功能通过路由元数据和页面角色判断控制显示。

6.10.2 请求封装

Axios 拦截器负责：

- 自动添加 Bearer Token。
- 识别业务码 200。
- 统一展示错误消息。
- HTTP 401 时清理 Token 并跳转登录页。
- HTTP 403 时提示无权限。

6.10.3 状态管理

Store	内容
user	Token、用户信息、登录和退出
app	侧边栏、主题和全局布局
tagsView	已访问页面标签

6.10.4 列表页面设计

- 搜索条件与分页状态独立保存。
- 删除后刷新当前页；当前页为空时回退上一页。
- 表格支持字段显隐。
- 异步文档状态可通过轮询刷新。

7. 数据设计

7.1 实体关系

```
sys_dept 1 --- N sys_user
sys_user 1 --- N doc_knowledge_base
sys_dept 1 --- N doc_knowledge_base
doc_knowledge_base 1 --- N doc_document
doc_document 1 --- N doc_chunk
sys_user N --- N doc_knowledge_base (收藏)
sys_user 1 --- N qa_conversation
doc_knowledge_base 1 --- N qa_conversation
qa_conversation 1 --- N qa_message
qa_message 1 --- 0..1 ticket_order
```

7.2 主要数据表

表	关键字段	关键索引
sys_user	username、password、role、dept_id、status	username 唯一，dept_id + status
sys_dept	name、parent_id、sort	parent_id + sort
doc_knowledge_base	name、owner_id、visibility、dept_id	owner_id + visibility + dept_id
doc_knowledge_base_favorite	user_id、kb_id	user_id + kb_id
doc_document	kb_id、title、file_name、parse_status	kb_id + parse_status + is_deleted
doc_chunk	doc_id、kb_id、chunk_index、embed_status	doc_id + chunk_index，embed_status
qa_conversation	user_id、kb_id、title	user_id + create_time

表	关键字段	关键索引
qa_message	conversation_id、question、answer、quality_score	conversation_id + create_time
ticket_order	user_id、kb_id、status、priority	user_id + status + create_time
stat_daily	stat_date	stat_date 唯一
stat_hot_question	question_hash、kb_id、ask_count	kb_id + question_hash
sys_operation_log	user_id、module、action、result	user_id + create_time

7.3 ID 与时间

- 主键使用雪花算法生成。
- Java Long 在 JSON 中序列化为字符串。
- 时间由应用层统一写入。
- 创建时间、更新时间、逻辑删除字段全局统一。

7.4 对象存储

Bucket	对象路径	用途
rag-documents	document/{uuid}.{ext}	文档原文件
user-avatar	avatar/{uuid}.{ext}	用户头像

预览使用预签名 URL，默认有效期 600 秒。

7.5 向量存储

元素	设计
Collection	kb_{kbId}
维度	1024
距离	Cosine
Point ID	分片 ID
Payload	知识库、文档、分片、内容摘要

8. 接口通用设计

8.1 路径与协议

- 接口前缀：/api/v1
- 数据格式：application/json; charset=UTF-8
- 文件上传：multipart/form-data
- 时间格式：yyyy-MM-dd HH:mm:ss
- 日期格式：yyyy-MM-dd

8.2 认证

Authorization: Bearer <token>

8.3 统一响应

```
{
  "host": null,
  "code": 200,
  "errorMessage": null,
  "data": {}
}
```

8.4 分页响应

```
{
  "code": 200,
  "errorMessage": null,
  "data": [],
  "currentPage": 1,
  "size": 10,
  "total": 25
}
```

8.5 通用校验

- $page \geq 1$
- $1 \leq pageSize \leq 100$
- $1 \leq topK \leq 20$
- 问题内容不能为空。
- ID 不能为空且必须属于当前用户可访问资源。

9. 接口详细设计

9.1 登录

请求

```
{
  "username": "zhangsan",
  "password": "123456",
  "rememberMe": false
}
```

响应

```
{
  "code": 200,
  "errorMessage": null,
  "data": {
    "token": "eyJhbGciOiJIUzI1NiJ9...",
    "user": {
      "id": "10001",
      "username": "zhangsan",
      "nickname": "张三",
      "role": 2,
      "deptId": "1001",
      "status": 1
    }
  }
}
```

9.2 注册

请求字段

字段	类型	必填	说明
username	string	是	唯一用户名
password	string	是	6-100 字符
nickname	string	否	显示名
email	string	否	邮箱
deptId	string	否	部门

注册接口不接收可信角色字段，服务端固定普通员工。

9.3 知识库查询

请求参数

参数	类型	必填	说明
page	integer	否	默认 1
pageSize	integer	否	默认 10
keyword	string	否	名称关键词
visibility	integer	否	可见范围

响应数据

```
{
  "id": "2001",
  "name": "研发知识库",
  "description": "研发规范",
  "ownerId": "10001",
  "visibility": 1,
  "deptId": "1001",
  "docCount": 18,
  "createTime": "2026-09-21 10:00:00"
}
```

9.4 创建知识库

```
{
  "name": "研发知识库",
  "description": "研发规范与接口文档",
  "icon": "book",
  "visibility": 1,
  "deptId": "1001"
}
```

9.5 上传文档

请求类型： multipart/form-data

参数	类型	必填
kbId	string	是
file	file	是

响应

- HTTP 202 或统一业务成功码。

- 返回文档 ID、初始状态和任务 ID。
- 前端根据文档列表状态轮询结果。

9.6 获取文档预览地址

```
{
  "previewUrl": "https://minio.example.com/rag-documents/document/uuid.pdf?X-Amz-...",
  "expireAt": 1789984800000
}
```

9.7 创建对话

```
{
  "kbld": "2001",
  "title": "接口咨询"
}

{
  "conversationId": "4001",
  "title": "接口咨询"
}
```

9.8 提问

```
{
  "conversationId": "4001",
  "question": "文档上传后多久可以检索？",
  "searchType": "vector",
  "topK": 5
}

{
  "messageId": "5001",
  "answer": "文档完成解析和向量化后即可检索。",
  "sources": [
    {
      "chunkId": "6101",
      "content": "文档上传后将异步解析并生成向量。",
      "score": 0.8734,
      "docId": "3001",
      "docTitle": "系统使用手册"
    }
  ]
}
```

9.9 提交反馈

```
{
  "messageId": "5001",
  "score": 1,
  "reason": "回答准确"
}
```

9.10 创建工单

```
{
  "title": "知识库没有检索到资料",
  "description": "该问题应存在于上传文档中。",
  "priority": 2,
  "messageId": "5001",
  "kbld": "2001"
}
```

9.11 解决工单

```
{
  "reply": "已补充文档并重新建立索引。",
  "status": 2
}
```

9.12 每日统计

查询参数

参数	类型	说明
startDate	date	开始日期
endDate	date	结束日期

响应元素

```
{
  "statDate": "2026-09-20",
  "totalQa": 128,
  "totalUsers": 35,
  "positiveRate": 92.50,
  "transferCount": 4,
  "avgResponseMs": 1860,
  "newDocs": 12
}
```

10. 接口错误设计

10.1 HTTP 状态

状态	含义
200	已处理，具体结果以业务码为准
400	请求参数错误
401	未认证或 Token 失效
403	无权限
404	资源不存在
409	状态冲突，例如重复处理工单
500	未处理的服务端异常

10.2 业务码

业务码	含义
200	成功
1	需要登录
2	密码错误
50	无效 Token
51	Token 过期
52	缺少 Token
500	缺少参数
501	参数无效
503	服务端错误

业务码	含义
1000	数据已存在
1002	数据不存在
3000	无权限操作
3001	需要管理员权限
3002	操作失败

10.3 异常处理

统一使用 @RestControllerAdvice：

- 参数异常转换为 400 和明确字段消息。
- 认证异常返回 401。
- 权限异常返回 403。
- 业务异常返回对应业务码。
- 未知异常记录请求 ID 和服务端日志，客户端只返回通用错误。

11. 安全设计

11.1 认证与授权

- Spring Security 无状态模式。
- 除白名单外 /api/v1/** 全部认证。
- 方法级权限使用 @PreAuthorize。
- 资源权限由统一权限服务判断。

11.2 密码与 Token

- 密码使用 BCrypt。
- JWT 密钥从密钥服务或环境变量读取。
- Token 中不存放敏感业务信息。
- 退出登录可加入 Redis 黑名单。

11.3 文件安全

- 扩展名、MIME 和文件头三重校验。
- UUID 重命名，不直接使用原始文件名作为对象路径。
- 上传目录和下载 URL 设置访问策略。
- 预览地址短时有效。

11.4 数据安全

- 所有写操作校验资源归属。
- 查询条件在服务端追加权限范围。
- 日志脱敏密码、Token、密钥和敏感请求体。

- 生产环境关闭 SQL 参数打印和异常堆栈返回。

12. 异步与定时任务设计

12.1 线程池

线程池	用途	建议配置
documentParseExecutor	文档解析	core 2, max 4, queue 100
embeddingExecutor	向量化	core 2, max 4, queue 200
indexExecutor	Qdrant 写入和删除	core 2, max 4, queue 200

外部服务调用为阻塞操作，解析与向量化使用独立线程池，避免相互占满。

12.2 任务状态

状态	含义
PENDING	等待执行
RUNNING	执行中
SUCCESS	成功
FAILED	失败
RETRYING	等待重试
DEAD	超过最大重试次数

12.3 重试策略

- 网络超时和 5xx 可重试。
- 参数错误、文件损坏和权限错误不自动重试。
- 重试间隔采用指数退避。
- 最大重试次数可配置。
- 失败任务保留错误信息和最后执行时间。

12.4 定时统计

- Cron: 0 0 2 * * ?
- 统计前一日数据。
- 使用唯一键实现幂等更新。
- 失败后记录任务状态并支持人工重跑。

13. 日志与监控设计

13.1 日志分类

日志	内容
访问日志	请求路径、用户、耗时、结果
业务日志	上传、解析、问答、工单和统计状态
外部调用日志	Embedding、Chat、Qdrant、MinIO 耗时和结果

日志	内容
异常日志	异常类型、堆栈、请求 ID
审计日志	管理员关键操作

13.2 监控指标

- 请求总量、错误率和 P95 耗时。
- 文档解析成功率和平均耗时。
- Embedding 调用成功率和耗时。
- Qdrant 写入和检索耗时。
- 问答平均响应时间。
- 失败任务数量。

13.3 健康检查

提供数据库、Redis、MinIO、Qdrant 和外部模型服务的健康状态检查，区分存活和就绪。

14. 部署设计

14.1 运行拓扑

```
Browser
-> Nginx
-> Spring Boot :8085
-> MySQL
-> Redis
-> MinIO
-> Qdrant
-> Embedding API
-> Chat API
```

14.2 配置隔离

配置	开发	生产
Profile	dev	prod
数据库	开发实例	环境变量或密钥服务
Redis	开发实例	环境变量或密钥服务
MinIO	开发实例	环境变量或密钥服务
AI API Key	本地安全配置	密钥服务
JWT 密钥	本地安全配置	密钥服务，定期轮换
日志	DEBUG	INFO/WARN，关闭敏感输出

14.3 发布流程

- 1 执行数据库迁移。
- 2 构建后端和前端产物。
- 3 部署后端，执行健康检查。
- 4 部署前端静态文件。

- 5 验证登录、知识库、文档和问答主链路。
- 6 出现异常时支持应用版本和数据库版本回滚。

15. 测试设计

15.1 单元测试

- JWT 生成、解析和过期判断。
- 密码注册和校验。
- 知识库权限算法。
- 文档上传文件校验。
- 文本分片边界。
- 工单状态流转。
- 统计指标计算。

15.2 集成测试

- MySQL CRUD 和分页。
- MinIO 上传、读取和预签名。
- Qdrant 创建、写入、检索和删除。
- Embedding 和 Chat API 异常处理。
- 文档完整处理链路。
- 删除和重新解析一致性。

15.3 接口测试

- 未登录、无权限、参数错误和正常请求。
- 资源越权访问。
- 分页边界和 Top-K 边界。
- 文件大小和类型限制。
- 工单重复处理冲突。

15.4 前端测试

- 路由守卫和角色入口。
- 登录、退出和 Token 过期。
- 列表筛选、分页和空状态。
- 上传状态轮询。
- 问答引用展示。

15.5 性能测试

- 100 个并发用户查询知识库和文档列表。
- 20 个并发用户上传 50 MB 文件。

- 5000 个分片的向量写入和检索。
- 单知识库 10 万向量的 Top-K 查询。

16. 实施顺序

16.1 第一阶段：基础能力

- 1 用户、认证和部门。
- 2 知识库及权限。
- 3 文档上传和元数据。
- 4 数据库迁移和基础部署。

16.2 第二阶段：RAG 主链路

- 1 文档解析和分片。
- 2 Embedding 和 Qdrant。
- 3 问答、引用和反馈。
- 4 删除和重解析一致性。

16.3 第三阶段：业务闭环

- 1 工单。
- 2 统计和热门问题。
- 3 工作台。
- 4 日志、监控和告警。

16.4 第四阶段：工程化与增强

- 1 自动化测试和 CI。
- 2 索引任务补偿和失败重试。
- 3 Rerank、混合检索和上下文预算。
- 4 分享、ACL、OCR 和多模型路由。

17. 设计追踪

需求域	对应设计章节
账号与用户	6.1、9.1、9.2、11
部门	6.2、7.2
知识库	6.3、7.2、9.3、9.4
文档	6.4、7.4、9.5、9.6
解析与索引	6.5、6.6、7.5、12
智能问答	6.7、9.7、9.8、9.9
工单	6.8、9.10、9.11
统计	6.9、9.12、12.4

需求域	对应设计章节
安全	11
部署与运维	13、14
测试	15

结论： 本软件设计说明书给出了 RAG KnowledgeNexus 的总体架构、模块边界、数据模型、关键流程和接口设计。编码实现应按照模块职责和接口契约推进，并通过测试验证权限、异步处理、索引一致性和 RAG 问答主链路。